

Table Tennis Serve Analysis

A Context-Aware Serve Recommendation Workflow

Shreyans Jain

2026-08-07

Table of contents

1 Overview	1
1.1 Project Snapshot	1
2 Motivation	2
3 Research Questions	2
4 Dataset	2
4.1 The Leakage Constraint	2
5 Analytical Design	3
6 Feature Engineering	3
6.1 Historical Combo Statistics and the Leakage Fix	3
7 Modeling Approach	4
7.1 Results	4
8 Exploratory Findings	4
9 Recommendation System	5
10 Repository Structure	5
11 Notebooks	6
12 Reproducing the Pipeline	6
13 Interpretation Guidelines	6
14 Limitations	7
15 Future Work	7
16 License	7

1 Overview

This project started with a pretty simple question I kept asking myself between points: given the score, who I'm playing, and the serves I actually have, which serve gives me the best shot at winning the point? Table Tennis Serve Analysis is my attempt to answer that with data instead of gut feel. It tracks serves at the point level and works through exploratory analysis, feature engineering, a few supervised models, and a recommendation step that ranks serve options for a given situation.

I want to be honest about what this is. The dataset is small and it comes from one player's matches, so nothing here proves that a serve "causes" a win. What it does is surface patterns and tendencies in how serves have performed. Read the results as decision support, not as table tennis law.

1.1 Project Snapshot

Area	Current State
Dataset size	500 serve-level observations

Area	Current State
Match coverage	8 matches against 8 opponents
Raw data	data/table_tennis_serves.csv
Processed data	data/processed/table_tennis_serves_features.csv
Modeling target	Binary point outcome: server won vs. lost
Best cross-validated model	LASSO logistic regression
Recommendation output	Ranked serve options with predicted value and reliability context
Validation approach	Match-aware grouped cross-validation

2 Motivation

Serving is the strongest part of my game, and it's also the one place where I have full control before the rally starts. But most of my serve choices came down to feel, memory, and whatever I happened to remember after a match. I wanted to turn that into something repeatable, so I started logging every serve along with the context around it and looking at which patterns actually lined up with winning points or forcing weak returns.

The goal isn't to replace the judgment you build over years of playing. It's to give myself a clearer picture of what has worked, against whom, and in what situations, so that between-point decisions have something concrete behind them.

3 Research Questions

1. Which serve characteristics line up most with winning the point?
2. How do serve outcomes change depending on the opponent's style and skill?
3. Which serves are most likely to set up the rally I actually wanted?
4. Does the score or late-game pressure change which serves look strongest?
5. Can a model rank serve choices using only what I know before the point starts?

4 Dataset

The raw dataset has one row per served point: 500 rows across 25 columns. Each row pulls together what I decided before the serve, the match context, some information about the opponent, and what happened after contact.

The columns fall into a few natural groups:

- Serve characteristics: `serve_type`, `spin_type`, `spin_intensity`, `serve_length`, `placement_zone`, `toss_height`, `contact_point`, `side`
- Match context: `match_id`, `game_number`, `server_score`, `receiver_score`, `game_state`
- Opponent profile: `opponent_id`, `opponent_skill_level`, `opponent_style`
- Return and rally diagnostics: `return_type`, `return_quality`, `return_placement`, `rally_length`, `point_end_type`
- Outcome and tactical intent: `point_outcome`, `intended_setup`, `rally_type_achieved`, `chop_rally_outcome`

4.1 The Leakage Constraint

One thing I had to be careful about is not letting the model cheat. The variables split cleanly into two kinds. First there are pre-point variables, the things I know before I serve, which are fair game for prediction and recommendation. Then there are post-serve variables that describe what happened after contact. Those are great for understanding a serve, but I can't use them to recommend a serve, because I don't have them yet when the decision gets made. Fields like `return_quality`, `rally_length`, and `point_end_type` fall into that second group.

There's also a sneakier leak hiding among the pre-point fields. A historical aggregate like a serve combo's win rate can leak the answer if a point's own outcome gets folded into the aggregate that describes it. I handle that explicitly, and I walk through it in Section 6.1.

5 Analytical Design

I ended up running the analysis on two tracks.

Track A is the decision-support side. It only uses information available before contact: the serve attributes themselves, the score state and pressure flags, the opponent's style and skill, and some engineered serve combinations along with their historical statistics. The thing it's trying to predict is whether the server won the point.

Track B is the explanatory side. It looks at why a serve might work, using things that only exist after the serve happens: return type and quality, return placement, rally length, how the point ended, whether the rally I intended actually showed up, and the chop-rally outcome when it applies. This track helps me interpret the mechanism, but none of it feeds the pre-point model.

6 Feature Engineering

The processed dataset takes the raw 25 columns up to 52, which works out to 27 engineered features, with no nulls in the engineered columns.

Feature family	Examples	Purpose
Outcome encoding	point_won	Binary modeling target
Score state	score_margin, is_tied, is_trailing, is_leading	Ahead, behind, or level
Late-game pressure	is_late_game, is_deuce_or_later, is_game_point_for_server, is_high_pressure	Distinct pressure states
Opponent indicators	opponent_is_looper, opponent_is_chopper, opponent_is_attacker, opponent_skill_numeric	Archetype and skill
Serve combinations	serve_spin_combo, serve_length_spin_combo, serve_placement_combo, full_serve_combo	Serves as combinations
Spin flags	is_heavy_spin, is_low_spin, spin_length_interaction	Discrete spin thresholds
Historical combo stats	combo_attempts, combo_win_rate, combo_reliability	Sample-size-aware history

6.1 Historical Combo Statistics and the Leakage Fix

combo_win_rate and combo_reliability summarize how a specific serve combination has done historically. The obvious way to build them is to average point_won across every time I used that combo. The problem is that “every time” includes the point you’re currently describing, which leaks the answer into its own feature. A combo I only tried once gets a win rate of exactly 0 or 1, which is just that single point's outcome wearing a disguise. Grouped cross-validation by match doesn't catch this either, because the same combo shows up in more than one match. The leak is at the row level, not the match level.

The fix is to compute both statistics leave-one-out. Each point's own outcome is left out of the aggregate that describes it, and combos with barely any history fall back to the dataset's overall win rate. Once I did that, `combo_win_rate` dropped out of the top 10 most-correlated features. Before the fix it was the single strongest predictor by a wide margin, which was a red flag. Afterward the models lean on a genuinely varied feature set instead of one near-tautological column.

7 Modeling Approach

The question I'm testing is whether the pre-point features beat a majority-class baseline at predicting the point outcome.

I evaluated four models: a majority-class baseline for a floor, a LASSO logistic regression for something sparse and interpretable, a random forest for nonlinear interactions and feature importances, and a gradient boosting classifier as a second nonlinear benchmark.

On validation, I deliberately avoided naive random row splits. Points from the same match are correlated, since they share an opponent, the tactical adjustments, the score dynamics, and the session itself. Splitting rows at random would leak match-specific information across the split. Instead the notebooks use grouped cross-validation with `GroupKFold` by `match_id` over 5 folds, so each validation fold looks more like a match the model hasn't seen.

7.1 Results

Model	Mean acc.	Acc. std.	Mean ROC-AUC	AUC std.
Baseline (majority class)	0.544	—	n/a	n/a
LASSO logistic regression	0.671	0.063	0.746	0.062
Random forest	0.644	0.023	0.696	0.083
Gradient boosting	0.617	0.051	0.646	0.069

LASSO came out on top, both on ROC-AUC and on Brier score, and it's the model I saved as the primary recommendation pipeline (`serve_win_probability_model.pkl`). All three real models clear the baseline, and LASSO does it by the largest and steadiest margin.

I'd read these numbers as a directional signal rather than a precise estimate. On 500 points across 8 matches, the fold-to-fold spread is not small compared to the gaps between models. Notebook 3 also reports bootstrap 95% confidence intervals on the pooled out-of-fold accuracy and ROC-AUC over 2000 resamples, and those intervals are wide and overlap across models. So the ranking is believable as a direction, but it isn't something I'd call statistically separable on this much data.

8 Exploratory Findings

A few patterns stood out in the exploratory work:

- Heavy spin lines up with better outcomes. In the EDA notebook it showed roughly a 17 percentage-point win-rate gap over light spin.
- The best serve type depends on who's across the table. It shifts between allround, blocker, defender, and looper opponents.
- Combinations tell you more than single attributes. Serve type on its own is often less informative than serve type together with placement, length, and spin.
- Sample size matters. Some of the flashiest win rates rest on only a handful of attempts, so the recommendation step can't just sort by raw win rate.

I'd treat all of these as patterns specific to this player and this sample until there's more data behind them.

9 Recommendation System

The recommendation step ranks candidate serves for a given match context. It blends the model's predicted win probability with the combo's historical performance and a correction for how much history that combo actually has, so a lightly-used serve doesn't get to punch above its evidence.

The score used in `4_Serve_Recommendation.ipynb` is:

```
Serve Score =  
    0.70 * predicted point-win probability    (from the trained model)  
+ 0.20 * historical combo win rate           (from observed match data)  
+ 0.10 * combo reliability                   (capped sample-size correction)
```

The predicted probability gets the most weight because it's the piece that actually reacts to context. It was trained on the serve attributes, the score state, and the opponent profile together. The historical win rate adds serve-level signal the model might miss, and the reliability term quietly pulls down combos with few attempts so a 2-attempt fluke can't outrank something I've tested plenty of times.

The notebook walks through a few concrete scenarios: a neutral point against a looper, a high-pressure deuce against an all-round opponent, and trailing against a defender, plus a sweep where I vary the score margin on its own to see how the ranking moves.

Two caveats are worth stating plainly. Because the model uses the whole feature set rather than one dominant column, its predicted probabilities cover the full 0 to 1 range, and a serve with almost no historical data can still land an extreme, overconfident-looking score. That's why the recommend step in `src/run_pipeline.py` filters to `combo_reliability >= 0.5`, roughly 15 or more historical attempts, by default. Pass `--min-reliability 0.0` if you want the full unfiltered ranking. The second caveat is that `combo_win_rate` at inference time comes from the same history the model trained on, not refreshed leave-one-out per candidate.

10 Repository Structure

```
Table-Tennis-Serve-Analysis/  
├─ README.md / LICENSE.md / requirements.txt  
├─ Makefile           # clean -> features -> train -> recommend  
├─ pytest.ini / ruff.toml / .gitignore  
├─ .github/workflows/ci.yml # ruff lint + pytest on push / PR  
├─ data/  
│   └─ table_tennis_serves.csv  
│   └─ processed/table_tennis_serves_features.csv  
├─ models/           # generated by `make train`; gitignored  
│   └─ lasso_model.pkl / rf_model.pkl / gb_model.pkl  
│   └─ serve_win_probability_model.pkl # primary model  
│   └─ model_features.pkl  
├─ notebooks/  
│   └─ 1_EDA.ipynb  
│   └─ 2_Feature_Engineering.ipynb  
│   └─ 3_Modeling.ipynb  
│   └─ 4_Serve_Recommendation.ipynb  
│   └─ 5_Model_Evaluation_and_Interpretation.ipynb  
├─ src/              # script versions of the pipeline  
│   └─ clean_data.py / features.py / train_model.py  
│   └─ recommend_serves.py  
│   └─ run_pipeline.py # CLI wiring the four steps together  
├─ tests/test_pipeline.py # leakage + smoke tests  
└─ reports/figures/
```

The `src/` directory holds standalone script versions of the notebook pipeline, covering cleaning, feature engineering, training, and recommendation, so the work can run outside a notebook, for example in a future dashboard or CLI tool. `src/run_pipeline.py` wires the four steps into one command, and the `Makefile` gives you `make all` (clean, features, train, recommend) along with `make test` and `make lint`. The model files under `models/` are not committed (`models/*.pkl` is gitignored), so run `make train` once after cloning to regenerate them before opening notebooks 4 or 5.

11 Notebooks

Notebook	Purpose
1_EDA.ipynb	Serve distributions, win rates, placement combinations, rally length, spin effects, opponent-style matchups
2_Feature_Engineering.ipynb	Score-state features, opponent encodings, serve-combination features, spin flags, interactions, leakage-safe reliability stats
3_Modeling.ipynb	Trains and evaluates baseline, LASSO, random forest, and gradient boosting with grouped validation
4_Serve_Recommendation.ipynb	Turns predictions and reliability features into ranked serve recommendations for example contexts
5_Model_Evaluation_and_Interpretation.ipynb	Stability check, coefficient interpretation, calibration curve, opponent-style breakdown, decile backtest

12 Reproducing the Pipeline

```
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
make train                    # regenerates models/*.pkl (gitignored)
```

The notebooks are meant to run in order, 1 through 5, since each one reads the previous notebook's output as it goes from raw CSV to processed CSV to saved model pipelines. The same pipeline runs as plain scripts through `make all` (or `python -m src.run_pipeline`). You can also hit the recommendation step directly for any context you like:

```
python -m src.run_pipeline recommend \
--server-score 10 --receiver-score 10 --game-state deuce \
--opponent-skill-level advanced --opponent-style looper \
--side backhand_side --top-n 10 --min-reliability 0.5
```

Anything you leave off falls back to the built-in example context.

13 Interpretation Guidelines

Since this is observational match data, I try to keep the language honest. I'd say "heavy spin is associated with a higher point win rate in this dataset," not "heavy spin causes a higher point win rate." Serve choices are tangled up with opponent quality, the score, what I was trying to set up, and how confident I felt at the time, so the model's output is evidence to weigh, not a verdict.

14 Limitations

- The dataset is small: 500 points across 8 matches.
- It's one player's serving, so it won't necessarily generalize.
- Some combinations have few attempts, which makes their raw win rates jumpy.
- Opponents adapt mid-match, so past performance may not predict future play.
- I'd want to validate on more matches before leaning on this for real match prep.
- Scores for combinations with little history can come out overconfident.

15 Future Work

- Grow the dataset to 100 or more matches and a wider range of opponent styles.
- Add video-linked labeling to validate serve quality and return classification.
- Generate cleaner plots under `reports/figures/` and surface the key ones.
- Put together a one-page tactical summary for match prep.
- Build a small Streamlit dashboard for interactive recommendations on top of the `src/` scripts.

16 License

MIT License. See `LICENSE.md` at the repository root. You're free to use, copy, modify, and distribute the code, with no warranty.